Technical Report

Project Title

Habit & Productivity Analytics API

Section 1. Introduction and Project Overview

The Habit & Productivity Analytics API is a data-driven REST API project that demonstrates how raw behavioural data can be transformed into structured, validated, and reusable software services. The project was designed to deliver a **correct, testable, and demonstrable system** rather than only a working prototype. It is based on a 90-day habit tracking dataset containing daily activity information such as workout duration and other productivity-related signals. This dataset was chosen to combine structured values with realistic data quality issues, which improves the validity of data cleaning, transformation, and API-oriented modelling decisions.

The objective was not only to implement CRUD operations, but to build an end-to-end workflow from raw CSV data to a usable application layer. In the final system, CSV data is cleaned and mapped into a relational schema, exposed through REST endpoints, aggregated through an analytics endpoint, and presented through a browser-based dashboard that consumes live backend responses.

The final implementation includes a FastAPI backend with full CRUD support for categories and records, JWT-based protection for write operations, an analytics summary endpoint, a pytest-based validation suite with 22 passing tests, and a frontend dashboard. The project also supports environment-based database configuration, allowing the same codebase to run locally with SQLite and in deployment with PostgreSQL. This architecture was chosen to improve reproducibility across environments and to ensure a realistic web-service workflow.

Section 2. System Architecture and Technology Choices

The system uses a layered architecture to separate responsibilities and improve maintainability. The routing layer defines HTTP endpoints and request handling rules. The CRUD layer contains the main application logic for database operations and analytics aggregation. Schema models define validated request and response contracts, while ORM models represent relational entities in the database. The database layer manages connection setup and session lifecycle.

A typical request flow is: client request → route handler → schema validation → CRUD logic → database read/write → JSON response. This architecture was prioritised to minimise duplicated logic, enforce early validation, and improve testability. The frontend dashboard functions as an API consumer rather than a separate logic layer, which ensures the backend remains the single source of truth for both CRUD data and analytics.

The technology stack was selected to match the needs of a coursework-scale API while still reflecting sound engineering practice. FastAPI was chosen for rapid development and built-in interactive documentation. SQLAlchemy provides structured ORM-based persistence, while the database configuration supports SQLite for local development and PostgreSQL for deployment via `DATABASE_URL`. This keeps local setup simple while supporting persistent hosted environments. Pydantic was chosen to

enforce strict validation of inputs and outputs. Pandas was chosen to simplify CSV parsing and preprocessing, and Pytest was chosen to provide repeatable evidence of correctness. These decisions improve consistency, reliability, reproducibility, and maintainability across the system.

Section 3. Data Source, Schema Mapping, and Import Pipeline

This project uses the **"90-Day Habit Tracker for Personal Growth"** dataset obtained from Kaggle (https://www.kaggle.com/datasets/uthaya1995/90-day-habit-tracker-for-personal-growth).
It was selected because it contains repeated daily records, mixed habit indicators, and realistic data quality issues such as missing values and inconsistent formats. This is well suited to a data-driven API project, where modelling quality depends directly on preprocessing quality.

The raw CSV data is mapped into two relational entities: `habit_categories` and `habit_records`. The first stores category-level metadata, while the second stores dated observations such as habit label, completion status, duration, category reference, and notes. This schema was designed to keep CRUD and analytics queries clear and maintainable.

To support safe deployment, the project also includes a seed tracking table (`seed_history`) and a guarded pre-deploy seed script. Instead of relying only on row counts, the script records a named seed marker after a successful import. On later deployments, that marker is checked before reseeding. This was designed to improve reliability and prevent duplicate imports.

Raw Field	Internal Field	Transformation Rule
`Date`	`record_date`	Parsed into ISO date format (`YYYY-MM-DD`); invalid values are skipped.
`Workout_Duration`	`duration_minutes`	Converted to numeric minutes and validated as non-negative when present.
`Journaling`	activity signal	Normalised into a boolean-style signal for rule-based classification.
`Reading_Time`	activity signal	Converted to a numeric indicator and used in rule-based classification.
`Sleep_Hours`	activity signal	Converted to a numeric value and retained as an input signal where available.
activity signals	category	Mapped into categories such as Fitness, Health, or Wellness using explicit rules.
multiple fields	`completed`	Inferred through threshold-based and signal-based decision rules.

Rule-based transformation was chosen as an explicit trade-off against more complex methods. A complex approach might increase modelling flexibility, but explicit rules were prioritised to improve transparency and reproducibility. Instead of opaque heuristics, the project uses mapping logic that can be explained, tested, and justified, which improves maintainability and assessment defensibility.

Section 4. API Design, CRUD Functionality, and Analytics

The API is centred on two main resources: `habit_categories` and `habit_records`. Each resource supports full CRUD operations, satisfying the coursework requirement for a complete database-backed CRUD model. Categories store grouping metadata, while records store individual observations linked through foreign keys.

The endpoint design follows standard REST conventions. `GET` retrieves resources, `POST` creates resources, `PUT` updates resources, and `DELETE` removes resources with clear status behaviour, including `204 No Content` for successful deletion. Responses are consistently returned as JSON. This was prioritised to ensure predictable behaviour, strong client usability, and stable integration.

Beyond CRUD, the API includes a real aggregation endpoint at `/analytics/summary`. This endpoint returns six metrics derived from persisted records: `total_records`, `completed_records`, `completion_rate`, `average_duration`, `records_per_category`, and `daily_trend`. These values are computed at the backend layer, which ensures consistency between storage, business logic, and presentation.

The frontend dashboard calls `/analytics/summary` during initial load and refresh actions. This provides evidence of end-to-end integration between data storage, API processing, and user-facing output.

Interactive API documentation (Swagger UI)

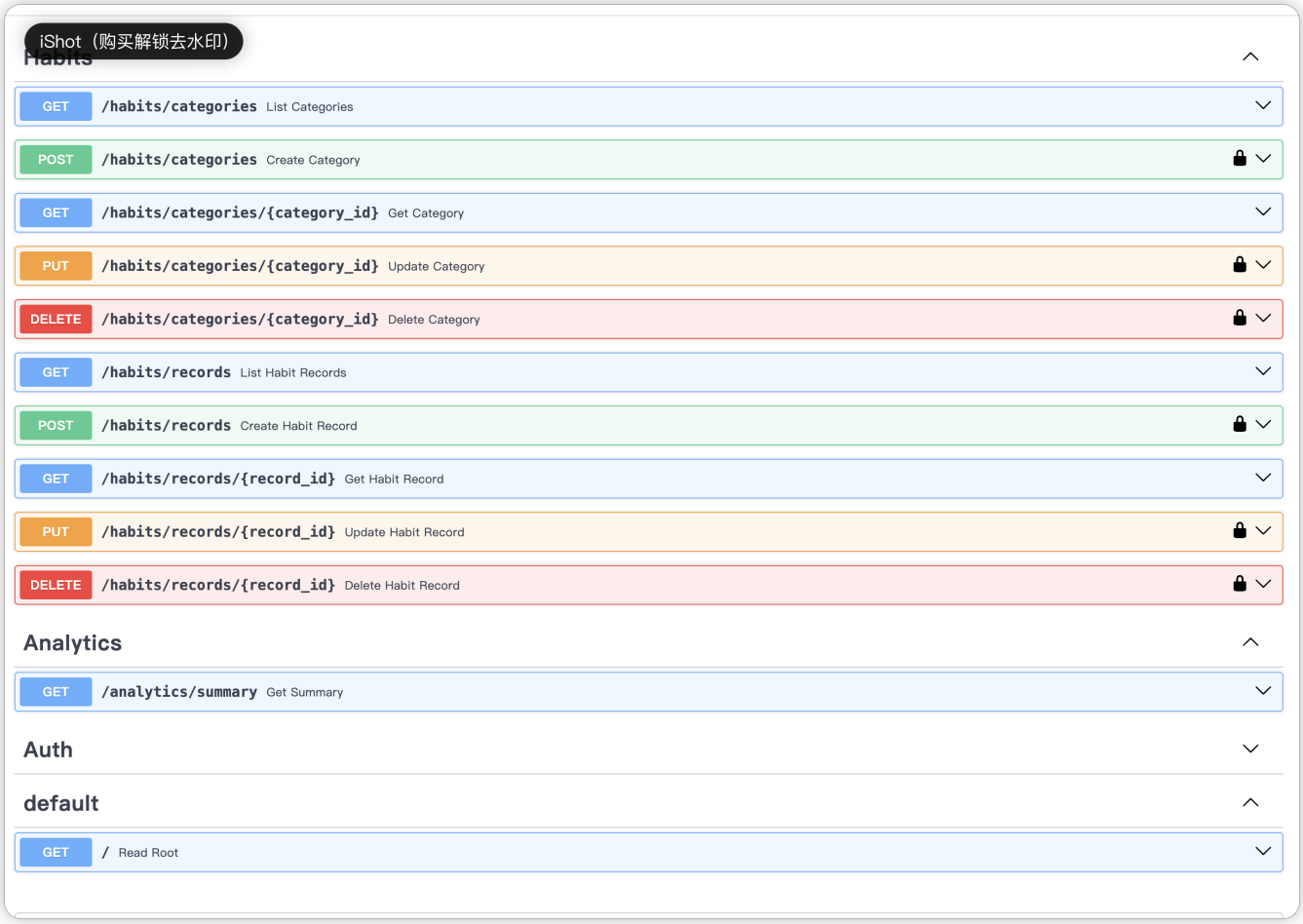

Figure 1. Interactive API documentation (Swagger UI) used for endpoint-level verification.

/analytics/summary response

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/analytics/summary' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/analytics/summary
```

Server response

Code	Details
200	Response body


```
{
  "total_records": 183,
  "completed_records": 117,
  "completion_rate": 63.93,
  "average_duration": 30.27,
  "records_per_category": [
    {
      "category_id": 1,
      "category_name": "Fitness",
      "count": 161
    },
    {
      "category_id": 5,
      "category_name": "Health",
      "count": 3
    },
    {
      "category_id": 2,
      "category_name": "Learning",
      "count": 1
    },
    {
      "category_id": 4,
      "category_name": "Wellness",
```


Figure 2. `/analytics/summary` response returned from the backend aggregation endpoint.

Section 5. Authentication, Error Handling, and Validation

Authentication is implemented using a JWT-based workflow. A client submits credentials to `/auth/login` (JSON client flow) or `/auth/token` (OAuth2 form flow for Swagger Authorize), receives a bearer token, and then includes that token in the `Authorization: Bearer <token>` header for protected requests. In this system, write operations on categories and records (`POST`, `PUT`, and `DELETE`) are protected, while read operations remain open for demonstration and analysis.

This security model was chosen as a trade-off between **implementation complexity** and **demonstrable security**. It provides meaningful access control while keeping scope appropriate for coursework constraints.

Error handling was designed to be predictable and client-friendly. Business-rule errors or invalid references return `400`, authentication failures return `401`, missing resources return `404`, and validation failures return `422`. The API returns errors in a standard `detail` + `code` structure, improving consistency for frontend handling and debugging.

Validation is enforced through Pydantic schemas before business logic is executed. Required fields, length constraints, and numeric boundaries are checked at the request boundary so malformed data does not propagate into the persistence layer. Together, JWT protection, structured errors, and schema validation improve reliability and maintainability.

Section 6. Testing and Validation

Testing was carried out at the API level using Pytest together with FastAPI's TestClient. The objective was to verify both expected behaviour and failure-path behaviour. The current result is fully passing and provides measurable evidence that the implemented features behave consistently under normal and exceptional conditions.

> Test Execution Evidence:
>
> ```
> 22 passed in 0.65s
> ```

This output provides reproducible evidence of system correctness.

The test suite covers authentication success and failure, unauthorized access to protected endpoints, CRUD flows for categories and records, duplicate category handling (`400`), invalid foreign-key references (`400`), missing resources (`404`), analytics endpoint behaviour, and schema boundary validation (`422`).

Automated tests were complemented by runtime checks through Swagger UI and helper scripts for API checks and database verification. This combined strategy was chosen to improve confidence in both backend correctness and real interactive behaviour.

Automated test execution result

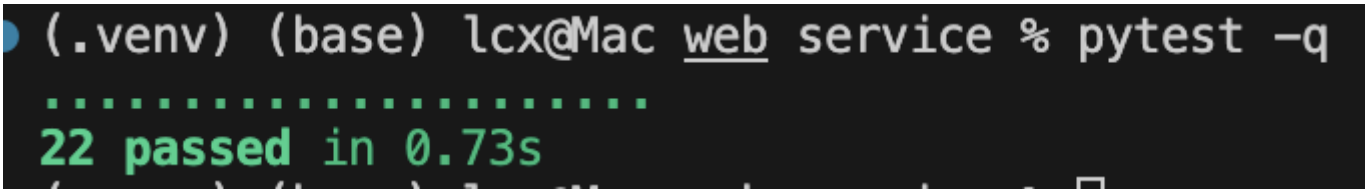

Figure 3. Automated test execution output showing all tests passed.

Section 7. Challenges and Lessons Learned

A major challenge was source-data quality. The CSV dataset did not map directly to the target schema, so import logic required explicit handling of missing values, type conversion, and rule-based interpretation. This provided evidence that data-driven API reliability depends heavily on transformation quality, not only endpoint implementation.

A second challenge was defining the `completed` field. A duration-only rule would have reduced implementation complexity, but would not represent the mixed dataset faithfully. A multi-signal rule-based approach was chosen to improve explainability while retaining practical accuracy.

A similar decision occurred in analytics placement. Computing summary metrics in the frontend would have been simpler initially, but would have weakened consistency and testability. Moving analytics computation to the backend improved data integrity and maintainability. Overall, this project demonstrates API design as **decision-making under imperfect data**.

Section 8. Limitations and Future Improvements

The current system has clear limitations. Authentication is intentionally lightweight and suitable for coursework demonstration rather than production deployment. User management is minimal, credentials

are fixed for demo purposes, and role-based access control is not implemented. The analytics endpoint remains summary-focused and does not yet include richer filters or comparative views. Although PostgreSQL support exists, versioned schema migration is not yet implemented.

Future work is framed as **production-ready next steps**. Priority upgrades include stronger user management and token controls, richer analytics filtering and comparative reporting, and deployment maturity through migration tooling, observability, and CI/CD enforcement for deployment scripts. These items were intentionally deferred to keep the submitted system focused, explainable, and demonstrable.

Section 9. GenAI Declaration and Reflection

GenAI tools were used in a supporting role rather than as a substitute for engineering judgement. They were used for ideation, debugging support, code refinement suggestions, and documentation drafting.

Tools Used and Purpose (Summary)

- **GitHub Copilot** was used during implementation to accelerate repetitive coding tasks, improve route and function structure, and reduce boilerplate.
- **ChatGPT** was used for design discussion, debugging analysis, and drafting or polishing documentation, including the README, API documentation, technical report, and GenAI declaration.

All key technical decisions were made by me, including schema design, data mapping rules, authentication scope, API structure, deployment logic, and validation strategy. Generated suggestions were accepted only after code-level and runtime validation.

In this project, GenAI improved speed and clarity, but ownership did not shift. **Final decisions remained under human control**, and final responsibility for correctness, coherence, and submission quality remained with me throughout.